NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 1: Course Launch, Computation, Python Basics, Variables, and Types

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 1 at a glance

Lecture 1

Course launch and computation

- course information;
- why engineers program;
- computation, algorithms, programs.

Lecture 2

Python basics and arithmetic

- running code and output;
- comments and errors;
- operators and precedence.

Lecture 3

Variables and types

- assignment and naming;
- integers, floats, strings, Booleans;
- first integrated calculator.

Problem → Steps → Code → Evidence → Meaning

NAZARBAYEV UNIVERSITY

LECTURE 1

Course Launch and Computational Thinking

Why engineers program, what computation means, and how an algorithm becomes a program

Welcome to Programming for Engineers

NAZARBAYEV UNIVERSITY

A quick diagnostic - no grades

Which statement best describes you today?

1. I have never written code.
2. I have tried a little programming.
3. I have completed a programming course.
4. I build programs regularly.

Show of hands - 90 s

Important

No prior programming experience is assumed. Curiosity, practice, and persistence matter more than speed in Week 1.

NAZARBAYEV UNIVERSITY

Week 1 learning goals

By the end of the week, you should be able to:

1. explain why computation and programming matter in engineering;
2. distinguish a problem, an algorithm, a program, and a numerical result;
3. run short Python statements and use Python as a calculator;
4. translate elementary formulas into Python expressions;
5. create meaningful variables and identify common data types;
6. check units and reasonableness before trusting an answer.

The target is not memorization

You are learning a repeatable way to turn an engineering question into a trustworthy computational result.

NAZARBAYEV UNIVERSITY

What is ENG 101 designed to do?

This course is

- an introduction to programming for practical engineering problems;
- suitable for students with no previous coding experience;
- a course in problem solving, coding, debugging, and communication;
- a foundation for later engineering study.

This course is not

- a competition to type code fastest;
- a collection of commands to memorize;
- only about obtaining an output;
- a substitute for mathematics, physics, or engineering judgment.

Course specification: introductory, engineering-focused, no prior experience required.

NAZARBAYEV UNIVERSITY

The semester journey



A transferable skill

Python is our first language. The underlying ideas - decomposition, testing, data, algorithms, and communication - transfer to MATLAB, C/C++, Julia, and other tools.

NAZARBAYEV UNIVERSITY

How the course is assessed



Assessments ask you to

read, trace, write, modify, debug, test, and explain code.

A running program is not enough

Correctness, clarity, units, reasoning, and individual understanding all matter.

Weights: final 40%, midterm 30%, laboratory 20%, assignment 10%.

NAZARBAYEV UNIVERSITY

Weekly rhythm and expected workload

Before class: skim the relevant pages and try the opening question.

In class: three lectures of about 50 minutes - predict, discuss, run, modify, and explain.

After class: reproduce examples, solve exercises, and record what still confuses you.

Course workload

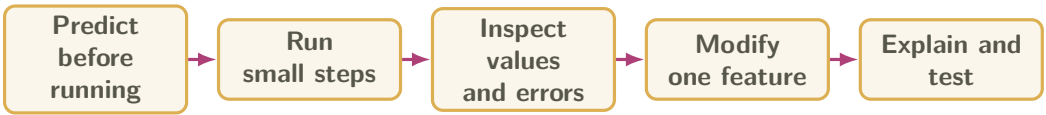
- 6 ECTS
- 45 contact hours
- 105 self-study hours
- 150 hours total

Plan ahead

The course specification advises approximately 10–12 hours of total work per week.

NAZARBAYEV UNIVERSITY

How we will learn programming



Collaboration

Discuss ideas and help one another learn when collaboration is permitted.

Accountability

Submitted work must reflect your own understanding. Follow the stated rules for AI tools and every assessment.

Why do engineers program?

Calculate

Repeat formulas reliably, convert units, and build design tables.

Model

Represent physical systems and explore how they change.

Communicate

Produce reproducible calculations, plots, reports, and evidence.

Analyze

Process measurements and identify trends or unusual values.

Compare

Evaluate alternatives, constraints, and trade-offs.

Automate

Let the computer handle scale while the engineer retains judgment.

Opening problem: energy use and cost

Workshop heater

A heater is rated at 1,500 W. It operates for 2 h. Electricity costs \$0.12 per kW h.

Find:

1. energy consumed in kW h;
2. operating cost;
3. how both change if the time becomes 5 h.

Before touching Python

- What are the inputs?
- What are the units?
- Which formulas are needed?
- What trend do you expect?

Think - pair - share, 3 min

Predict before computing

Reasoning without code

$$E_{\mathrm{kWh}} = P_{\mathrm{kW}} t_{\mathrm{h}}, \qquad C = E_{\mathrm{kWh}} c.$$

- 1,500 W = 1.5 kW
- Energy should be 3 kW h
- Cost should be \$0.36

Scaling check

If operating time changes from 2 h to 5 h while everything else stays fixed, energy and cost should be multiplied by

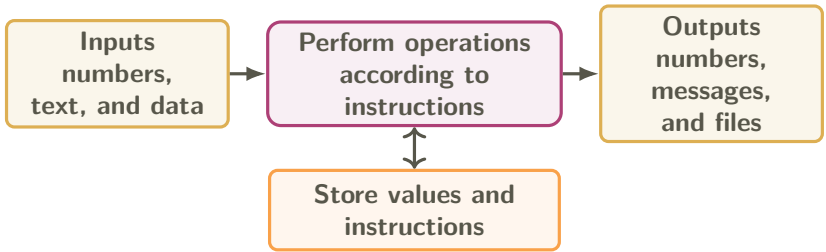
$$\frac{5}{2} = 2.5.$$

Why predict?

A prediction gives us evidence for checking the program later.

NAZARBAYEV UNIVERSITY

What does a computer actually do?

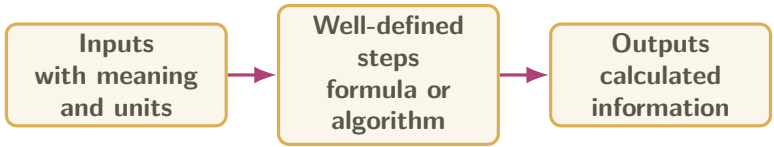


The limitation

Speed and memory do not supply meaning. The computer follows the instructions; the engineer decides whether the instructions and results are appropriate.

NAZARBAYEV UNIVERSITY

What is computation?



Working definition

Computation transforms inputs into outputs by following a well-defined sequence of steps.

Engineering example

Inputs: force F and area A; step: evaluate $\sigma = F/A$; output: stress σ - then check units, assumptions, and design significance.

NAZARBAYEV UNIVERSITY

Knowing what is different from knowing how

Declarative knowledge - “what”

$$y = \sqrt{x} \quad \Longleftrightarrow \quad y^2 = x,\ y \geq 0.$$

This defines a square root, but it does not tell a computer how to find one.

Imperative knowledge - “how”

1. choose a guess g;
2. check whether g^2 is close to x;
3. improve the guess;
4. repeat until accurate enough.

Computational thinking

Turn a desired relationship or outcome into an executable procedure.

NAZARBAYEV UNIVERSITY

What makes a useful algorithm?

1. **Inputs are specified.**
2. **Steps are precise.**
3. **The order is clear.**
4. **The process terminates.**
5. **Outputs are defined.**
6. **The method handles intended cases.**
7. **The result can be checked.**

Algorithm versus code

An algorithm describes the method. Code expresses that method precisely in a programming language.

NAZARBAYEV UNIVERSITY

Activity: write the algorithm before the code

Task

Write pseudocode for the heater problem. Your steps must include:

- storing the given quantities;
- converting W to kW;
- calculating energy and cost;
- printing quantities with units;
- one check of the result.

Pairs - 4 min

Do not write Python yet

Use plain language. Another student should be able to follow the steps without asking what you meant.

Possible first line

```
Store the heater power in
watts.
```

From pseudocode to a program

Pseudocode

1. store power in W;
2. convert W to kW;
3. store time and price;
4. calculate energy;
5. calculate cost;
6. display a report.

```
power_w = 1500
power_kw = power_w / 1000

time_h = 2
price_per_kwh = 0.12

energy_kwh = power_kw * time_h
cost = energy_kwh * price_per_kwh

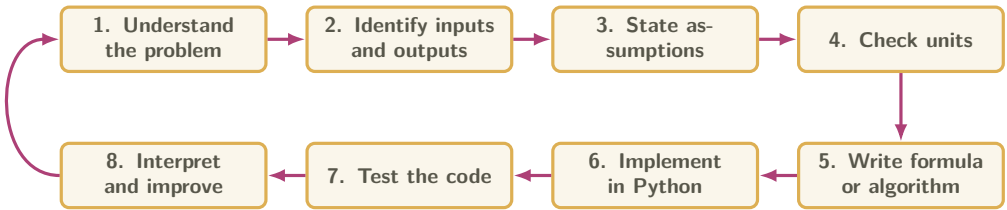
print(energy_kwh)
print(cost)
```

Precision matters

A computer executes the program you wrote, not the intention you had in mind.

NAZARBAYEV UNIVERSITY

The engineering-computation cycle



The central habit of the course

Syntax tells Python what to do. Engineering reasoning tells us what should be done, how to check it, and what the result means.

Your first Python instruction

```
print("Welcome to Programming for Engineers")
```

Read it aloud

Call the function `print` and give it one text value to display.

Live coding - 3 min

```
Welcome to Programming for
    Engineers
```

Try a deliberate mistake

Remove one quotation mark. Run the code and read the error message.

NAZARBAYEV UNIVERSITY

Lecture 1 recap and exit ticket

Key ideas

- computation transforms inputs through defined steps;
- an algorithm is a precise procedure;
- a program expresses an algorithm for execution;
- engineering computation includes units, testing, and interpretation.

Submit before leaving

In two sentences:

1. What can Python calculate?
2. What must the engineer still decide?

Individual - 2 min

NAZARBAYEV UNIVERSITY

LECTURE 2

Python Basics and Mathematical Operations

Run short programs, translate formulas, and learn to predict before pressing Run

NAZARBAYEV UNIVERSITY

Lecture 2 learning goals

By the end of this lecture, you should be able to:

1. distinguish an interactive shell, notebook cell, and script;
2. use `print()` and comments in short programs;
3. recognize syntax, runtime, and logic errors;
4. use Python arithmetic operators correctly;
5. apply precedence and parentheses when translating formulas;
6. use scientific notation and preserve engineering units.

Today we learn by doing

Most slides contain something to predict, run, modify, or debug.

NAZARBAYEV UNIVERSITY

Why Python is a useful first engineering language

Accessible now

- readable syntax;
- immediate feedback;
- useful as a calculator;
- short programs can solve real problems.

Powerful later

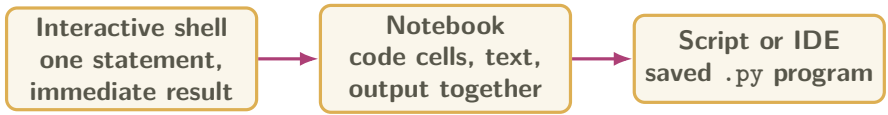
- numerical computing;
- data analysis and plotting;
- simulation and automation;
- machine learning and scientific libraries.

Transferable goal

The course is not about collecting every Python feature. It is about learning how to construct, test, and explain computational solutions.

NAZARBAYEV UNIVERSITY

Where does Python code run?



The shared habit

Write a small amount, run it, inspect the output, and correct mistakes.

Course environment

Use the Python 3 environment specified on Moodle. The course specification identifies Anaconda/Spyder as supported software.

Statements, values, and output

```
print("ENG 101")
print(3 + 2)
print("Voltage =", 12, "V")
```

```
ENG 101
5
Voltage = 12 V
```

Vocabulary

A **statement** instructs Python to do something. An **expression** is evaluated to produce a value. `print()` displays values.

Predict each output before running

NAZARBAYEV UNIVERSITY

Comments document engineering meaning

```
speed_m_per_s = 20       # m/s
time_s = 15              # s

distance_m = speed_m_per_s * time_s
print(f"Distance = {distance_m:.1f} m")
```

Useful comments explain

- units;
- assumptions;
- why a conversion is made;
- limitations or warnings.

Better than repeating the code

```
# convert mm^2 to m^2 before calculating stress
```

Three kinds of trouble

Syntax error

```
print("Stress)
```

Python cannot parse the program.

Runtime error

```
area = 0
stress = 5000 / area
```

Execution starts, then stops.

Logic or unit error

```
area_m2 = area_mm2 / 1000
```

The code runs but the engineering result is wrong.

Most dangerous

A plausible-looking wrong answer can be more dangerous than a visible crash.

NAZARBAYEV UNIVERSITY

Python as a calculator

Operator	Meaning	Example
+	addition	3 + 2 $\rightarrow 5$
-	subtraction	3 - 2 $\rightarrow 1$
*	multiplication	3 * 2 $\rightarrow 6$
/	true division	3 / 2 $\rightarrow 1.5$
//	floor division	7 // 3 $\rightarrow 2$
%	remainder	7 % 3 $\rightarrow 1$
**	exponentiation	3 ** 2 $\rightarrow 9$

Classic beginner mistake

In Python, powers use **. The symbol ^ is not exponentiation.

Predict - run - explain

```
print(5 + 2 * 3)
print((5 + 2) * 3)
print(10 / 4)
print(10 // 4)
print(10 % 4)
print(2 ** 3 ** 2)
```

Before running

Write six predicted outputs. Beside each one, explain which operation Python performs first.

Individual 2 min - then compare

NAZARBAYEV UNIVERSITY

Predict - run - explain

```
print(5 + 2 * 3)
print((5 + 2) * 3)
print(10 / 4)
print(10 // 4)
print(10 % 4)
print(2 ** 3 ** 2)
```

Before running

Write six predicted outputs. Beside each one, explain which operation Python performs first.

Individual 2 min - then compare

Answers

`11, 21, 2.5, 2, 2, 512`

Why do /, //, and % all exist?

True division

$$17/5 = 3.4$$

Use when the fractional result matters.

Whole groups

$$17//5 = 3$$

Three complete groups of five.

Remainder

$$17\%5 = 2$$

Two items remain after grouping.

Engineering examples

Batching samples, scheduling maintenance cycles, converting total seconds into minutes and remaining seconds, or checking periodic patterns.

Order of operations

1. parentheses ( )
2. exponentiation **
3. multiplication and division
4. addition and subtraction

Best practice

Use parentheses whenever they make the engineering formula clearer - even when Python would not require them.

Kinetic energy

$$KE = \frac{1}{2}mv^2$$

```
kinetic_energy = 0.5 * mass * velocity**2
```

Average of three measurements

```
average = (x1 + x2 + x3) / 3
```

Translate the formula, not just the symbols

Mathematics

$$\bar{x} = \frac{x_1 + x_2 + x_3}{3}$$

$$P = \frac{V^2}{R}$$

$$\sigma = \frac{F}{A}$$

```
average = (x1 + x2 + x3) / 3
power_w = voltage_v**2 / resistance_ohm
stress_pa = force_n / area_m2
```

Translation checklist

Check every numerator, denominator, exponent, and parenthesis. Then check units.

Scientific notation and readable numbers

```
yield_strength_pa = 250e6
area_m2 = 4.0e-4
speed_of_light_m_per_s = 3.0e8

one_million = 1_000_000
```

Interpretation

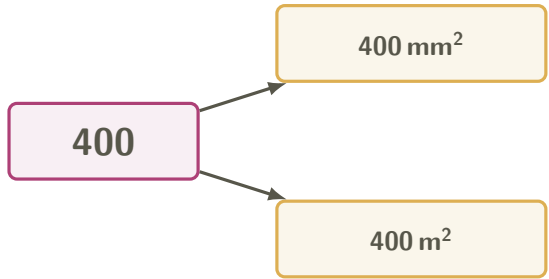
- `250e6` means 250×10^{6};
- `4.0e-4` means 4.0×10^{-4};
- underscores improve readability but do not change the value.

Quick check

`125e6 / 1e6` evaluates to `125.0`: the same stress expressed in MPa instead of Pa.

NAZARBAYEV UNIVERSITY

Python stores numbers - not physical units



Same number, radically different physical meaning

Keep units visible in variable names, comments, prompts, formulas, and printed output. Python cannot detect a unit mismatch unless we design the program to do so.

Live coding: energy and cost calculator

```
power_w = 1500                    # W
power_kw = power_w / 1000         # kW

time_h = 2                       # h
price_per_kwh = 0.12              # dollars/kWh

energy_kwh = power_kw * time_h
cost = energy_kwh * price_per_kwh

print(f"Energy used = {energy_kwh:.2f} kWh")
print(f"Cost = ${cost:.2f}")
```

Expected output

```
Energy used = 3.00 kWh
Cost = $0.36
```

Check

Does the output agree with the prediction from Lecture 1?

Modify, predict, and explain

Modify the calculator for each case:

1. time changes from 2 h to 5 h;
2. power changes from 1500 W to 2000 W;
3. electricity price becomes zero;
4. power doubles while time is halved.

For each case

1. predict the direction of change;
2. run the program;
3. explain the result in one sentence.

Pairs - 6 min

Debug this program

```
power_w = 1500
time_h = 2

energy_kwh = power_w * time_h
cost = energy_kwh * 0.12

print("Energy = energy_kwh kWh")
print(f"Cost = ${cost:.2f}")
```

Find at least two problems

One problem is about units. The other is about displaying a variable rather than literal text.

Individual 2 min - class repair

Debug this program

```
power_w = 1500
time_h = 2

energy_kwh = power_w * time_h
cost = energy_kwh * 0.12

print("Energy = energy_kwh kWh")
print(f"Cost = ${cost:.2f}")
```

Find at least two problems

One problem is about units. The other is about displaying a variable rather than literal text.

Individual 2 min - class repair

Core repairs

Convert W to kW before multiplying by hours, and use an f-string to insert `energy_kwh`.

Lecture 2 recap and exit ticket

You should now be able to

- run and inspect short Python code;
- use arithmetic operators;
- translate formulas with correct precedence;
- use comments and scientific notation;
- distinguish visible code errors from hidden engineering errors.

Exit ticket

Write the correct Python expression for

$$y = \frac{a + b}{c^2}$$

and name one unit check you would perform before trusting it.

Individual - 2 min

NAZARBAYEV UNIVERSITY

LECTURE 3

Variables and Data Types

Give engineering quantities meaningful names, understand assignment, and reason about the kinds of values Python stores

NAZARBAYEV UNIVERSITY

Lecture 3 learning goals

By the end of this lecture, you should be able to:

1. explain a variable as a name associated with a value;
2. distinguish Python assignment from algebraic equality;
3. choose valid, meaningful variable names with visible units;
4. identify `int`, `float`, `str`, `bool`, and `None`;
5. use `type()`, `int()`, `float()`, and `str()`;
6. recognize type-related and floating-point surprises;
7. build and test a small engineering calculation program.

NAZARBAYEV UNIVERSITY

Give engineering quantities meaningful names

Hard to audit

$$c = a/b$$

```
a = 12.0
b = 4.0
c = a / b
```

Engineering meaning is visible

$$I = V/R$$

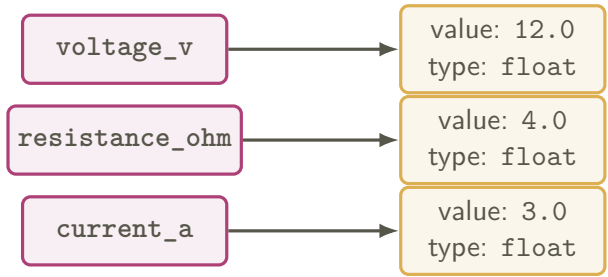
```
voltage_v = 12.0
resistance_ohm = 4.0
current_a = voltage_v / resistance_ohm
```

Names are part of technical communication

Another engineer should be able to understand what a value represents and which unit is expected.

NAZARBAYEV UNIVERSITY

A variable is a name associated with an object



Important mental model

The variable name is not a box containing a permanent number. Assignment associates or re-associates a name with a value.

Assignment is not algebraic equality

In algebra

$$x = x + 1$$

has no finite solution.

In Python

```
x = 4
x = x + 1
print(x)
```

Output: 5

Read assignment from right to left

Evaluate the expression on the right using current values; then associate the name on the left with the result.

NAZARBAYEV UNIVERSITY

Trace reassignment step by step

```
temperature_c = 20
print(temperature_c)

temperature_c = temperature_c + 5
print(temperature_c)
```

Step	Value	Action
1	20	assign
2	20	print
3	25	reassign
4	25	print

Predict both outputs first

Why this matters

Reassignment is the basis of counters, loops, time stepping, and simulations.

Rules for Python variable names

A name may

- contain letters, digits, and underscores;
- begin with a letter or underscore;
- use uppercase or lowercase letters.

A name may not

- start with a digit;
- contain spaces;
- be a Python keyword;
- be treated as case-insensitive.

Valid

`force`, `force_n`, `area_m2`, `temperature_initial`

Invalid

`2force`, `beam length`, `class`

NAZARBAYEV UNIVERSITY

Meaningful names prevent engineering mistakes

Ambiguous

```
area = 400
area = area / 1000000
stress = force / area
```

Auditable

```
area_mm2 = 400
area_m2 = area_mm2 / 1_000_000
stress_pa = force_n / area_m2
```

Recommended style

Use lowercase `snake_case`. Include units in names when it prevents confusion, especially when several unit systems appear in the same program.

NAZARBAYEV UNIVERSITY

Common Python data types

int

`8`

counts, indexes, and number of bolts

float

`9.81`

measurements and calculated quantities

str

`"steel"`

text, labels, units, and messages

bool

`True`

yes/no and pass/fail states

None

`None`

absence of a useful value

Why types matter

A type determines what operations make sense for a value and how Python behaves when you use it.

Ask Python with `type()`

```
number_of_bolts = 8
beam_length_m = 2.5
material = "steel"
is_safe = True
result = None

print(type(number_of_bolts))
print(type(beam_length_m))
print(type(material))
print(type(is_safe))
print(type(result))
```

Predict

Write the five type names in order before running the program.

Think - 90 s

Ask Python with `type()`

```
number_of_bolts = 8
beam_length_m = 2.5
material = "steel"
is_safe = True
result = None

print(type(number_of_bolts))
print(type(beam_length_m))
print(type(material))
print(type(is_safe))
print(type(result))
```

Predict

Write the five type names in order before running the program.

Answer

`int, float, str, bool, NoneType`

Think - 90 s

NAZARBAYEV UNIVERSITY

Integers and floating-point numbers

`int`

Whole numbers such as `0`, `-3`, and `1250`.

- number of samples;
- number of bolts;
- loop counters and indexes.

`float`

Real-valued approximations such as `3.0`, `9.81`, and `4e-4`.

- length, time, mass;
- voltage and temperature;
- stress, pressure, speed.

Useful rule for beginners

Most measured or calculated engineering quantities should be represented as floats, even when a particular value happens to look whole.

NAZARBAYEV UNIVERSITY

Types change the meaning of operators

```
print(3 + 4)
print("3" + "4")
print(3 * 4)
print(3 * "A")
print("A" * "B")
```

Predict

Which lines perform arithmetic, which manipulate text, and which line raises an error?

Key point

The symbol alone does not determine the operation; the operand types also matter.

NAZARBAYEV UNIVERSITY

Types change the meaning of operators

```
print(3 + 4)
print("3" + "4")
print(3 * 4)
print(3 * "A")
print("A" * "B")
```

Predict

Which lines perform arithmetic, which manipulate text, and which line raises an error?

Results

`7`, `"34"`, `12`, `"AAA"`, then a `TypeError`.

Key point

The symbol alone does not determine the operation; the operand types also matter.

Boolean values represent engineering questions

```
stress_mpa = 120
allowable_stress_mpa = 150

is_within_limit = stress_mpa <=
    allowable_stress_mpa

print(is_within_limit)
print(type(is_within_limit))
```

Output

```
True
<class 'bool'>
```

Preview of Week 2

Branching lets a program act differently when a Boolean condition is `True` or `False`.

Input arrives as text

```
voltage = input("Enter voltage in V: ")
print(type(voltage))
```

Even if the user types 12, `input()` returns the string `"12"`.

```
voltage_v = float(
    input("Enter voltage in V: ")
)
```

Common conversions

`int(value)`, `float(value)`, and `str(value)`.

Clear prompts include units

Prefer `"Enter area in mm^2: "` over `"Enter area: "`.

NAZARBAYEV UNIVERSITY

A first floating-point surprise

```
result = 0.1 + 0.2
print(result)
print(f"Result = {result:.2f}")
```

```
0.30000000000000004
Result = 0.30
```

Why?

Many decimal fractions do not have an exact finite representation in binary, so a float stores a close approximation.

Do not panic - but remember

Formatting changes display, not the stored value. Later we will compare floating-point results using tolerances rather than exact equality.

NAZARBAYEV UNIVERSITY

Integrated example: stress from force and area

```
force_n = 8000.0                    # N
area_mm2 = 400.0                    # mm^2

area_m2 = area_mm2 / 1_000_000
stress_pa = force_n / area_m2
stress_mpa = stress_pa / 1_000_000

print("AXIAL STRESS REPORT")
print("-" * 28)
print(f"Force = {force_n:.1f} N")
print(f"Area = {area_mm2:.1f} mm^2")
print(f"Stress = {stress_mpa:.2f} MPa")
```

Expected result

$\sigma = 20\,\text{MPa}$.

Assumption

The load is axial and the stress is treated as uniform over the cross-section.

Sanity trend

Doubling force should double stress; doubling area should halve stress.

NAZARBAYEV UNIVERSITY

Bug hunt: syntax, name, type, or unit?

```
Force_n = "8000"
area_mm2 = 400
area_m2 = area_mm2 / 1000

stress_pa = force_n / area_m2
print("Stress = stress_pa Pa")
```

Classify each problem

- case-sensitive name mismatch;
- number stored as a string;
- wrong area conversion;
- printing literal text instead of a value.

Pairs - 4 min

NAZARBAYEV UNIVERSITY

Test with simple cases before realistic ones

Known-case test

For

$$F = 10\,\mathrm{N}, \qquad A = 2\,\mathrm{m}^2,$$

the program should return

$$\sigma = 5\,\mathrm{Pa}.$$

Trend tests

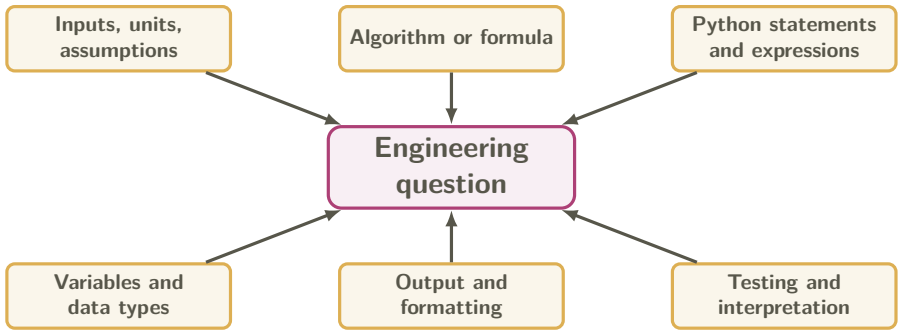
- $F \times 2 \Rightarrow \sigma \times 2$;
- $A \times 2 \Rightarrow \sigma / 2$;
- positive $F, A \Rightarrow$ positive stress;
- incompatible units should trigger suspicion.

Testing is planned checking

A plausible result is not evidence by itself. Build evidence before trusting the program.

NAZARBAYEV UNIVERSITY

Week 1 concept map



The thread connecting all three lectures

Code is one part of an engineering solution: formulate, calculate, test, interpret, and communicate.

NAZARBAYEV UNIVERSITY

Week 1 exit ticket and preparation for Week 2

Exit ticket

Without running Python, explain the output or error:

1. `x = 3; x = x + 2; print(x)`
2. `print("3" + "2")`
3. `print(3 + "2")`
4. `print((2 + 3) * 4**2)`

Before Week 2

- rerun all examples from a clean session;
- complete the assigned Chapter 1–2 practice;
- bring one error message you encountered;
- preview strings, comparisons, branching, and iteration.

Individual - 3 min

Week 1 quick reference

Core syntax

`print(value)`	display a value
`# comment`	note for human readers
`name = value`	assignment
`+ - * /`	arithmetic
`// % **`	floor, remainder, power
`type(value)`	inspect a type

Engineering habits

- state inputs, outputs, and assumptions;
- keep units visible;
- predict sign, scale, and trend;
- test a simple known case;
- interpret the result physically;
- save a working version.

NAZARBAYEV UNIVERSITY

Suggested reading and source basis

- Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026.
- Y. Wang, *Programming for Engineers with Python*, working draft, August 2026, Chapters 1–2 and instructor notes.
- J. V. Guttag, *Introduction to Computation and Programming Using Python: With Application to Understanding Data*, 2nd ed., MIT Press, Chapters 1–2.

Primary preparation for students

Read the assigned sections and run every short example. Change one value, predict the effect, and rerun from a clean session.

NAZARBAYEV UNIVERSITY

Welcome to computational engineering

Ask precise questions. Write clear instructions. Test the result. Explain what it means.

ENG 101 - Programming for Engineers